Polymorphism and casting

Lecture 24

Based on Slides of Dr. Norazah Yusof

POLYMORPHISM

- A feature of object-oriented languages whereby the same method call can lead to different behaviors depending on the type of object on which the method call is made.
- Example : method walk()
  - person.walk();
  - cat.walk();
  - snake.walk();
  - lizard.walk()

POLYMORPHIC method

- A method that behaves differently when it is invoked on different objects.
- An overridden method, such as the

  `toString()` method, is an example of a polymorphic method because its use can lead to different behaviors depending upon the object on which it is invoked.

Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {
  public static void main(String[] ar)
{
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }

  public static void m(Object x) {
   System.out.println(x.toString());
  }
}

class GraduateStudent extends Student {
}

class Student extends Person {
  public String toString() {
    return "Student";
  }
}

class Person extends Object {
  public String toString() {
    return "Person";
  }
}
```

Method m takes a parameter of the Object type. You can invoke it with any object.

An object of a subtype can be used wherever its supertype value is required. This feature is known as *polymorphism*.

When the method m(Object x) is executed, the argument x's toString method is invoked. x may be an instance of GraduateStudent, Student, Person, or Object. Classes GraduateStudent, Student, Person, and Object have their own implementation of the toString method. Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as *dynamic binding*.

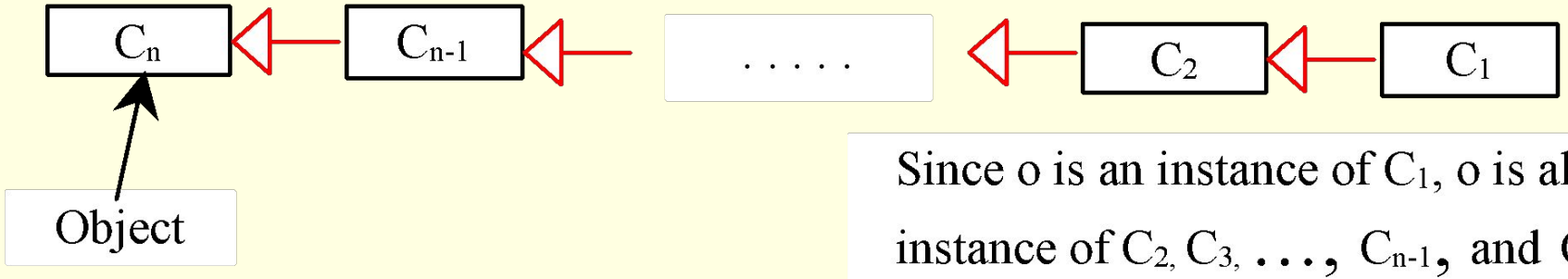
Dynamic Binding

Dynamic binding works as follows: Suppose an object o is an instance of classes C_1, C_2, ..., C_{n-1}, and C_n, where C_1 is a subclass of C_2, C_2 is a subclass of C_3, ..., and C_{n-1} is a subclass of C_n.

That is, C_n is the most general class, and C_1 is the most specific class. In Java, C_n is the Object class.

If o invokes a method p, the JVM searches the implementation for the method p in C_1, C_2, ..., C_{n-1} and C_n, in this order, until it is found.

Once an implementation is found, the search stops and the first-found implementation is invoked.



Since o is an instance of C_1, o is also an instance of C_2, C_3, . . . , C_{n-1}, and C_n

Generic Programming

```
public class PolymorphismDemo {
  public static void main(String[] args){
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }

  public static void m(Object x) {
   System.out.println(x.toString());
  }
}

class GraduateStudent extends Student {
}

class Student extends Person {
  public String toString() {
    return "Student";
  }
}

class Person extends Object {
  public String toString() {
    return "Person";
  }
}
```

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as **generic programming**.

If a method's parameter type is a superclass (e.g., Object), you may pass an object to this method of any of the parameter's subclasses (e.g., Student or String).

When an object (e.g., a Student object or a String object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., toString) is determined dynamically.

Casting (primitive type)

- **You can always assign a value to a numeric variable whose type supports a larger range of values;**
- **You cannot assign a value to a variable of a type with a smaller range unless you use *type cast* - convert a value of one type into another type.**
- **Casting a variable of a type with a small range to a variable of a type with a larger range is known as *widening a type*.**
- **Casting a variable of a type with a large range to a variable of a type with a smaller range is known as *narrowing a type*.**
- ***Widening a type* can be performed automatically without explicit casting (implicitly).**
- ***Narrowing a type* must be performed explicitly.**

Type Casting

```
Implicit casting
  double d = 3; (type widening)

Explicit casting
  int i = (int)3.0; (type narrowing)
  int i = (int)3.9; (Fraction part is truncated)
```

What is wrong?

i. **int x = 5 / 2.0;**
ii. **int i = 1;**
    byte b = i;

range increases →

byte, short, int, long, float, double

Casting Objects

Casting can also be used to convert an object of one class type to another within an inheritance hierarchy. In the preceding section, the statement

```
m(new Student());
```

assigns the object new Student() to a parameter of the Object type.

This statement is equivalent to:

```
Object o = new Student(); // Implicit casting
m(o);
```

The statement Object o = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object.

Why Casting Is Necessary?

Suppose you want to assign the object reference `o` to a variable of the `Student` type using the following statement:

```
Student b = o;
```

A compilation error would occur. Why does the statement Object o = new Student() work and Student b = o doesn't?

This is because a `Student` object is always an instance of Object, but an Object is not necessarily an instance of `Student`. Even though you can see that `o` is really a Student object, the compiler is not so clever to know it.

To tell the compiler that `o` is a `Student` object, use an explicit casting.

```
Student b = (Student)o; // Explicit casting
```

Casting from Superclass to Subclass

- **If a superclass object is not an instance of the subclass, a runtime ClasscastException occurs.**
- **To ensure that the object is an instance of another object before attempting a casting, use the operator instanceof.**

```
Object myObject =  new Circle();
    :
if (myObject instanceof Circle)
{
  System.out.println ("The diameter is " +
    ((Circle)myObject.getDiameter());
  :
 }
```

Casting from Superclass to Subclass

- **It always possible to cast an instance of a subclass to a variable of a superclass (known as *upcasting*), because an instance of a subclass is always an instance of its superclass.**
- **Casting an instance of a superclass to a variable of its subclass (known as *downcasting*), explicit casting must be used to confirm your intention to the compiler.**

Self Test 1

- **Assume that `circle1` is a `Circle object` and `object1` is a `GeometricObject`, are the following Boolean expressions true or false?**

1. `(circle1 instanceof GeometricObject)True`
2. `(object1 instanceof GeometricObject)F`
3. `(circle1 instanceof Circle)F`
4. `(object1 instanceof Circle)T`

Self Test 2

- **Are the following statement correct?**

1. ```
   Circle circle1 = new Circle(5);
   GeometricObject object1 = circle1;
   ```

2. ```
   GeometricObject object2 = new
                          GeometricObject();
   Circle circle1 = (Circle)object2;
   ```

3. ```
   GeometricObject object3 = new Circle();
   Circle circle3 = (Circle)object3;
   ```
   ```

The `protected` Modifier

- **The `protected` modifier can be applied on data and methods in a class. A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.**
- **private, default, protected, public**

Visibility increases →

```
private, none (if no modifier is used), protected, public
```

Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	–
default	✓	✓	–	–
private	✓	–	–	–

Visibility Modifiers

package p1;

```
public class C1 {
  public int x;
  protected int y;
  int z;
  private int u;

  protected void m() {
  }
}
```

```
public class C2 {
  C1 o = new C1();
  can access o.x;
  can access o.y;
  can access o.z;
  cannot access o.u;

  can invoke o.m();
}
```

package p2;

```
public class C3
          extends C1 {
  can access x;
  can access y;
  can access z;
  cannot access u;

  can invoke m();
}
```

```
public class C4
          extends C1 {
  can access x;
  can access y;
  cannot access z;
  cannot access u;

  can invoke m();
}
```

```
public class C5 {
  C1 o = new C1();
  can access o.x;
  cannot access o.y;
  cannot access o.z;
  cannot access o.u;

  cannot invoke o.m();
}
```

A Subclass Cannot Weaken the Accessibility

- **A subclass may override a protected method in its superclass and change its visibility to public (strengten the accessibility).**
- **However, a subclass cannot weaken the accessibility of a method defined in the superclass.**
- **For example, if a method is defined as public in the superclass, it must be defined as public in the subclass.**

NOTE

The modifiers are used on classes and class members (data and methods), except that the <u>final</u> modifier can also be used on local variables in a method. A final local variable is a constant inside a method.

The `final` Modifier

- **The `final` class cannot be extended:**

```
final class Math {
  ...
 }
```

- **The `final` variable is a constant:**

```
final static double PI = 3.14159;
```

- **The `final` method cannot be overridden by its subclasses.**

The equals() and hashCode() Methods in the Object Class

- The equals() method compares the contents of two objects.
- The hashCode() method returns the hash code of the object. Hash code is an integer, which can be used to store the object in a hash set so that it can be located quickly.

The `equals` Method in the Object Class

The `equals()` method compares the contents of two objects. The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {
  return (this == obj);
}
```

For example, the equals method is overridden in the Circle class.

```
public boolean equals(Object o) {
  if (o instanceof Circle) {
    return radius == ((Circle)o).radius;
  }
  else
    return false;
}
```

NOTE

The == comparison operator is used for comparing two primitive data type values or for determining whether two objects have the same references.

The equals method is intended to test whether two objects have the same contents, provided that the method is modified in the defining class of the objects.

The == operator is stronger than the equals method, in that the == operator checks whether the two reference variables refer to the same object.

Optional

The hashCode() method in the Object Class

Invoking hashCode() on an object returns the hash code of the object. Hash code is an integer, which can be used to store the object in a hash set so that it can be located quickly.

Hash sets will be introduced in Chapter 22, "Java Collections Framework."

The hashCode implemented in the Object class returns the internal memory address of the object in hexadecimal.

Your class should override the hashCode method whenever the equals method is overridden. By contract, if two objects are equal, their hash codes must be same.

Optional

The finalize, clone, and getClass Methods

- The finalize method is invoked by the garbage collector on an object when the object becomes garbage.
- The clone() method copies an object.
- The getClass() method returns an instance of the java.lang.Class class, which contains the information about the class for the object.
  - Before an object is created, its defining class is loaded and the JVM automatically creates an instance of java.lang.Class for the class.
  - From this instance, you can discover the information about the class at runtime.

Hiding Fields and Static Methods

You can override an instance method, but you cannot override a field (instance or static) or a static method.

If you declare a field or a static method in a subclass with the same name as one in the superclass, the one in the superclass is hidden, but it still exists.

The two fields or static methods are independent. You can reference the hidden field or static method using the <u>super</u> keyword in the subclass.

The hidden field or method can also be accessed via a reference variable of the superclass's type.